

ARM PrimeCell

PS2 Keyboard/Mouse Interface (PL050)

Integration Manual

ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Integration Manual

© Copyright ARM Limited 1999. All rights reserved.

Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

Document confidentiality status

This document is Open Access (no restriction on distribution).

Product status

The information in this document is Final (information on a developed product).

Web address

<http://www.arm.com>

Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Feedback on the ARM PrimeCell PS2 Keyboard/Mouse Interface

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

Contents

1	ABOUT THIS DOCUMENT	1
1.1	Change history	1
1.2	References	1
1.3	Terms and abbreviations	1
2	SCOPE	2
3	INTRODUCTION	2
4	INTEGRATION	3
4.1	Design Flow Supported	3
4.2	Signal Connectivity	4
4.3	Bus Interconnection	5
4.4	Endianness	5
4.5	Customisation	6
4.6	Synthesis	6
4.6.1	Setup Scripts	6
4.6.2	Command Script	6
4.6.3	Constraint Scripts	7
4.7	Integration with AMBA Micropack components	8
4.8	Interrupt Connection	8
4.9	Clocking	9
4.10	Clock Tree synthesis	10
4.11	Resets	11
4.12	Reset Buffering	12
4.13	Synchronisers	13
4.14	Test Methodologies	13
4.14.1	AMBA TIC methodology	13
4.14.2	Scan Insertion and ATPG	14
4.15	Functional and Timing Verification	15

1 ABOUT THIS DOCUMENT

1.1 Change history

Issue	Date	Change
A	7 January, 1999	First release.
B	4 June, 1999	Second release, Verilog testbenches added.

1.2 References

This document refers to the following documents.

Ref	Doc No	Title
1	ARM IHI 0011	AMBA Specification (Rev 2.0)
2	ARM DDI 0143	ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual
3	PL050 DDES 0000	ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Design Manual
4	ARM DDI 0138	Example AMBA System Technical Reference Manual
5	ARM DUI 0092	Example AMBA System User Guide
6	ARM DAI 0005	AMBA Interconnection schemes Application Note 05

1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

Term	Meaning
AHB	Advanced High-performance Bus
AMBA	Advanced Micro-controller Bus Architecture
APB	Advanced Peripheral Bus
ASB	Advanced System Bus
ASIC	Application Specific Integrated Circuit
ATPG	Automatic Test Pattern Generation
IP	Intellectual Property
KMI	Keyboard/Mouse Interface
RTL	Register Transfer Level
STA	Static Timing Analysis

2 SCOPE

This document describes how the ARM PrimeCell PS2 Keyboard/Mouse Interface (KMI) may be integrated into an AMBA-based ASIC when using a typical industry standard ASIC design flow.

3 INTRODUCTION

The ARM PrimeCell PS2 Keyboard/Mouse Interface (KMI) is a reusable soft-IP block that has been developed with the prime aim of reducing time to market for ASIC development.

A carefully chosen set of design rules have been followed to allow faster integration in most ASIC design flows using standard synthesis and test tools. The implementation can easily be customised to suit specific customer requirements.

For further information on AMBA, refer to the AMBA Specification [Ref.1].

For detailed technical information and programming data on the PrimeCell KMI block, refer to the ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual [Ref.2].

The ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Design Manual [Ref.3] contains information about the implementation and design of the PrimeCell KMI block that may be needed for optional customisation of this block.

The Example AMBA System Technical Reference Manual [Ref.4] and User Guide [Ref.5] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA. Further information can also be found on use of TICTalk test vectors for production test.

The AMBA Interconnection schemes Application Note 05 describes on-chip bus interconnection using OR bus, multiplexer and tri-state within an AMBA system [Ref.6].

4 INTEGRATION

4.1 Design Flow Supported

The ARM PrimeCell KMI has been designed to allow integration with other IP blocks using typical ASIC design flows. Although consideration has been given to minimisation of area (gate count) and power-consumption, ease of integration has been given higher priority to aid design reuse.

The following sections of this document address issues that may arise during ASIC system-level integration of the ARM PrimeCell KMI block.

An example ASIC design flow is described below:

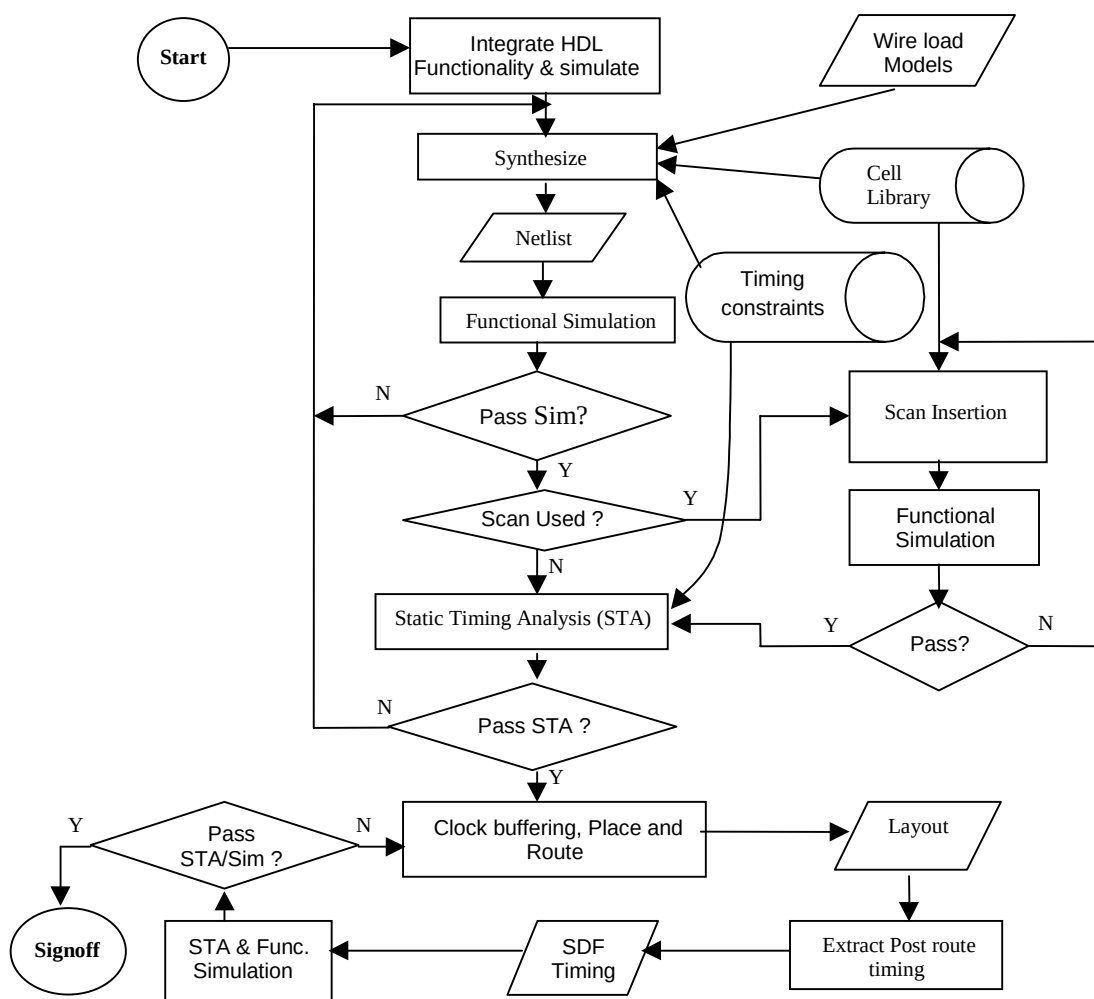


Figure 4.1-1: Example ASIC Design Flow

As an alternative to dynamic functional simulation it is also possible to use logic equivalence checking.

4.2 Signal Connectivity

The following tables describe the I/O pins for the ARM PrimeCell KMI block. For further information refer to ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Technical Reference Manual [Ref.2].

Signal Name	Type	Source / Destination	Description
PCLK	IN	Clock Generator	APB clock input used to time all bus transfers.
BnRES	IN	Reset Controller	Bus reset input signal (active low).
PADDRH[7:6]	IN	APB Bridge	Subset of APB address bus.
PADDRL[4:2]	IN	APB Bridge	Subset of APB address bus.
PWDATA[7:0]	IN	APB Bridge	Subset of unidirectional APB write data bus. Data is written into the peripheral via this bus.
PRDATA[7:0]	OUT	APB Bridge	Subset of unidirectional APB read data bus. Data is read out of the peripheral via this bus.
PENABLE	IN	APB Bridge	APB enable signal, asserted HIGH for one cycle of PCLK to enable a bus transfer.
PWRITE	IN	APB Bridge	When HIGH, this signal indicates a write into the peripheral and when LOW, a read from the peripheral.
PSEL	IN	APB Bridge	When HIGH, this signal indicates that the peripheral has been selected by the APB Bridge.

Table 4.2-1: AMBA APB Signals

Signal Name	Type	Source / Destination	Description
KMIREFCLK	IN	Clock Generator	KMI reference clock input.
nKMIRST	IN	Reset Controller	Reset signal to the KMIREFCLK domain (active low).
KMITXINTR	OUT	Interrupt Controller	KMI transmit interrupt.
KMIRXINTR	OUT	Interrupt Controller	KMI receive interrupt.
KMIINTR	OUT	Interrupt Controller	KMI interrupt. Combined interrupt generated as a logical OR function of the two individual interrupt sources.
SCANMODE	IN	Test Controller	KMI scan test mode input.

Table 4.2-2: On-Chip Signals

Signal Name	Type	Source / Destination	Description
KMICLKIN	IN	PAD	KMI clock input
KMIDATAIN	IN	PAD	KMI serial data Input
nKMIDATAEN	OUT	PAD	KMI data pad output enable (active low)
nKMICKEN	OUT	PAD	KMI clock pad output enable (active low)

Table 4.2-3: Signals to I/O pad cells

4.3 Bus Interconnection

The KMI block interfaces with the AMBA APB interface using separate uni-directional read (output) and write (input) data buses PRDATA and PWDATA. For further information refer to the AMBA Specification [Ref.1].

The data bus may be interfaced at the APB Bridge either using multiplexers or logical OR'ing of data buses from other APB peripheral blocks. In common with other APB peripheral blocks, the KMI block forces all read data bus lines low when there are no APB read transfers in progress.

However, it is also possible to interface the KMI block with a 3-state bi-directional data bus. A wrapper needs to be added around the KMI block connecting the read and write data buses to bi-directional drivers. Control of the 3-state drivers can be done using the APB control signals. A bi-directional 3-state implementation, while yielding a small area saving, will make system-level static timing analysis more difficult, as well as causing problems in scan test/ATPG and achieving high coverage fault grading.

4.4 Endianness

Although the KMI is designed for a system using little-endian memory organisation, it is equally possible to integrate the block in a system using big-endian memory organisation.

Since the KMI has an 8-bit wide APB data interface it can be connected in a little endian system to bits 7:0 of the data bus and use byte-wide accesses for maximum performance.

For a big-endian system, to maximise the system performance, the KMI block should be connected to bits 31:24 of the data bus. All byte accesses to the peripheral will then appear at an address offset of zero. The zero address offset then allows simple generation of the register address in ARM code using an 8-bit shifted constant. A byte load is preferred to a half-word or word load since software would need to mask out the unused bits. However, this method of connection in a big-endian system requires a script to transpose the bytes of the 32-bit wide TicTalk test vectors so that they will be applied correctly to the KMI block now connected to bits 31:24 of the data bus, unless scan testing is used instead of AMBA TIC testing.

Alternatively, in a big-endian system, the KMI block could be connected to bits 7:0 of the data bus and only word accesses used for this block. Since the unused bits will be undefined, software must now mask them out making the code less efficient. The TicTalk test vectors no longer need to have the byte order transposed however.

4.5 Customisation

The ARM PrimeCell KMI can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Information to support such modification is included in the ARM PrimeCell PS2 Keyboard/Mouse Interface (PL050) Design Manual [Ref.3].

4.6 Synthesis

The KMI block has been synthesised using the Synopsys Design Compiler synthesis tool with the scripts located in the `synopsys_ref` directory.

A README text file in the `synopsys_ref/` directory provides details of the synthesis directory structure used.

There are two master compile scripts. The script `kmi.cmd` synthesizes the KMI to the chosen target cell library and a separate script `kmi_scan.cmd` scan inserts the mapped design which has already been created by the `kmi.cmd` script.

The final area and gate count that will be achieved is dependent on the target cell library used, and synthesis timing constraints and command options.

The synthesis scripts consist of the following three main categories:

4.6.1 Setup Scripts

Scripts that are common to several designs are located in the `synopsys_ref/scr_common` directory. The following files need several parameters or variables to be changed in order to target a different cell library.

The setup script, `setup.scr` defines the following:

- target, link and symbol libraries
- UNIX directory path for synopsys read and write caches

The `global.scr` script uses parameters from the `cb25v2.1settings.scr` script, and these parameters need modifying to define:

- Library worst and best case condition names
- Default cell drive and load strength

The `cb25dontuse.scr` script is included by the `setup.scr` script, to specify which cells in the target library should not be used. It is recommended that an equivalent file be created for the target library as per library vendor or customer requirements. It is also a practice to sometimes prevent the synthesis tool from inferring lower drive cells for timing reasons. The user must ensure that the `cb25dontuse.scr` file is modified and renamed appropriately for the chosen technology library and included by the `setup.scr` script.

The `cell_lib` directory must contain the files, or links to the target cell library files, in Synopsys format.

4.6.2 Command Script

The master compile command script `kmi.cmd` in the `synopsys_ref/scr_kmi` directory analyses the HDL source before mapping and optimising to a specific target technology cell library. The `kmi.cmd` script includes setup and constraint scripts to set timing and design rule constraints on the design, prior to optimisation. The compiled design is written out both in native Synopsys `.db` format and in VHDL/Verilog netlist format. Pre-route (post-synthesis) timing information for back-annotation is written out to SDF files and various reports for timing and violations are also written out. Additionally the run-time log is redirected into a separate `log/` directory.

The compile script `kmi_scan.cmd` in the `scr_scan` directory performs scan insertion for the design, reading in the Synopsys intermediate format (.db) file created by the `kmi.cmd` script. Output is again created for either VHDL or Verilog netlist and SDF output files, as well as a Synopsys intermediate format (.db) file.

Prior to synthesis, several variables need to be set. The scripts use environment variables which are assigned to synthesis variables in the scripts. The environment variables provide flexibility without requiring editing of the scripts. The following environment variables are used:

- \$PERIPH is assigned to `design_root` to define the top-level directory for the design.
- \$HDL_IN is assigned to `hdl` to define the input HDL source. HDL_IN should be set to either `vhdl` or `verilog` in order to read in the VHDL or Verilog source RTL files.
- \$HDL_OUT is assigned to `hdl_out` to define the output format for netlist and SDF files. It can be set to either `vhdl`, `verilog` or both to generate either VHDL, Verilog or both VHDL and Verilog output files.

If the environment variables are not required, then the scripts can be modified to directly assign values.

4.6.3 Constraint Scripts

- a) Operating conditions and signal load/drive values should be set for the chosen technology library in the `global.scr` file.
- b) Timing constraints for APB bus signals are set via the `apb_params.scr` and the `apb_slave.scr` file. As delivered, the timings are set for a 50 MHz clock. These timings should be adjusted according to the target frequency required. The `apb_params.scr` file has been written using parameters and only requires amendment of the clock frequency with all other bus parameters (except clock skew) scaling accordingly.

Clock skew needs to be set explicitly as minus (worst case) and plus (best case) uncertainty. The following default values have been used as preliminary values prior to layout:

- the minus uncertainty value is set at 5% of the clock period.
- the plus uncertainty value is set at 40% of the minus uncertainty value.

The plus uncertainty derives from best case condition and it is less than the minus uncertainty value which derives from worst case condition.

The timing parameters assume a bus timing budget of 20% for output delays and 40% for input setup. This is a reasonable estimate assuming that the process technology is such that APB Bridge can meet this constraint with signals being available after 40% of the clock period.

If a very high frequency bus clock is being used, it is important to ensure that the read data output timing is adequate to meet the requirements of the APB Bridge. In some designs the read data may not be sampled by the APB Bridge, but instead passed directly to the system bus. In this case, it is necessary to ensure that the timing requirements are met for the block which is sampling the data.

The value of 40% of clock period allowed for input setup of APB inputs, applies for a style of APB Bridge design with registered inputs implementing logic after the registers. If the APB Bridge has registered outputs then the input setup can be increased, which should result in a reduction of area required.

- c) Timing constraints for non-AMBA KMI-specific signals are set via the `kmi.scr` file. The `KMIREFCLK` frequency is set to be same as that of `PCLK` for synthesis. Sample input and output delay constraints are also specified for the other signals. It should be noted that for asynchronous signals, the timing budgets are arbitrary since they can never be accurate. The minimum delays are set so that the synthesis tool does not insert buffers to fix hold violations that do not exist in reality.

4.7 Integration with AMBA Micropack components

The KMI module is designed to be integrated into an AMBA system via the APB (Advanced Peripheral Bus.) It is designed to work in systems where an AMBA bus master can read from or write to the KMI through the ASB-to-APB bridge in response to interrupt signals. In the majority of systems, DMA capability will not be required for a low bandwidth peripheral such as the KMI. It can be efficiently controlled, directly by the CPU.

An example APB Bridge is available within the ARM Micropack product. The decode logic within the APB Bridge will need to be altered to take into account the address location of the KMI and any other APB peripherals in the memory map. The base address of the KMI needs to be defined on a system specific basis and the bridge decode logic altered accordingly. The APB Bridge appears as a slave peripheral on the ASB with a decode signal, which must be altered in the HDL code of the AMBA decoder module, to select the correct range for the APB peripherals (including the KMI). If AMBA TIC methodology is to be used for production testing then any change of address location for the KMI block must also be reflected in the AMBA TIC test vectors, by amending the base address declaration for the KMI production test program.

4.8 Interrupt Connection

The KMI module provides flexibility in choice of an interrupt handling strategy by providing both individual and combined interrupts. Two interrupting conditions (receive interrupt request, transmit interrupt request) are indicated by separate interrupt lines. These conditions are also logically OR'ed together to provide a single combined interrupt.

The interrupt architecture could use one of the two schemes below:

- 1 Each peripheral provides a single combined interrupt source to the system interrupt controller, which is the OR function of the internal masked sources. The system interrupt controller may then provide another level of masking on a per peripheral basis.
- 2 Each peripheral passes all of its individual sources to the system interrupt controller which would also provide the mask registers for each individual source. In this way a global interrupt service routine would be able to read the entire set of sources from one wide register in the system interrupt controller.

Figure 4.8-1 illustrates how a typical peripheral block outputs both the individual masked interrupts as well as a combined output.

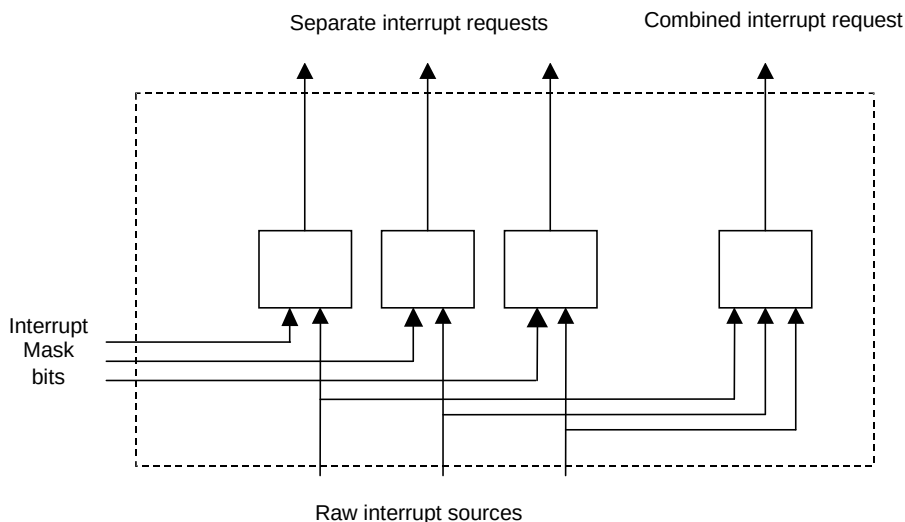


Figure 4.8-1: Interrupt structure

The former system has the advantage of allowing modular device drivers which always know where to find the interrupt source control register bits. The latter system has an advantage where a more integrated interrupt service routine is required for the system and it would be especially attractive where the time to read from the peripheral registers is significant compared to the CPU clock speed in a real time system.

The KMI block has been designed to support system architectures which use either type of interrupt controller since the overhead for doing so is small.

Software may be implemented with either separate interrupt service routines for each interrupt or with a single interrupt service routine which polls each of the two interrupt status bits to determine the interrupt source. Each of these interrupt sources can be masked individually within the KMI itself, thus complementing the mask bits provided in the interrupt controller. The ARM core has two interrupts – nIRQ and nFIQ. Normally a low priority peripheral block such as the KMI would route the interrupt source to the nIRQ input of the ARM core.

4.9 Clocking

The ARM PrimeCell KMI HDL contains two clock sources at the top level – PCLK and KMIREFCLK. There are two possible clocking options, depending on the required configuration:

- Clock both sources with the AMBA system clock, BCLK/HCLK. This can be done if the Bus Clock frequency is suitable for creation of the required KMI bit clock rates using the divider implemented in the KMI. Use of a single clock source allows easier static timing analysis and the use of a single scan chain (if scan insertion is being used). This is the recommended approach for this design.

The APB Bridge could also be designed so that PCLK is an integer sub-multiple of BCLK/HCLK if a lower frequency bus clock is required in order to reduce power consumption.

- Clock PCLK and KMIREFCLK from separate sources. This is required when the system bus clock frequency is not suitable for generating the required bit clock rates. If separate asynchronous clocks are used, each clock domain can be individually statically analysed and the simplest strategy is to use two separate scan chains with no mixing of clocks or edges. It is also possible to implement a single scan chain mixing different clocks and scan tool vendor documentation should be consulted for further details.

It should be noted that the KMI design assumes that both the clock sources are free-running during normal device operation. While gating the clocks is not recommended, it might have to be employed in very power critical applications. It is imperative that the clocks are shut off only after the KMI is completely idle. Accesses to the KMI are not guaranteed to complete correctly if either clock is not free-running. Clocks should only be turned off when the RXBUSY and TXBUSY bits in the status register indicate that receive and transmit operations have completed.

Additionally, if clock gating is required, it should be implemented in a separate block to ensure ease of integration of the KMI block in a typical design flow.

In production test mode, both the clock sources should be driven from a single source, the APB clock PCLK. This is necessary to ensure that the TicTalk test vectors are frequency independent. It is also essential that the two clock trees PCLK and KMIREFCLK are balanced relative to each other in order to minimise clock skew. If KMIREFCLK is multiplexed as in figure 4.9-1 then it is important to ensure that a valid path exists from the PCLK source point to the input of the multiplexer for KMIREFCLK.

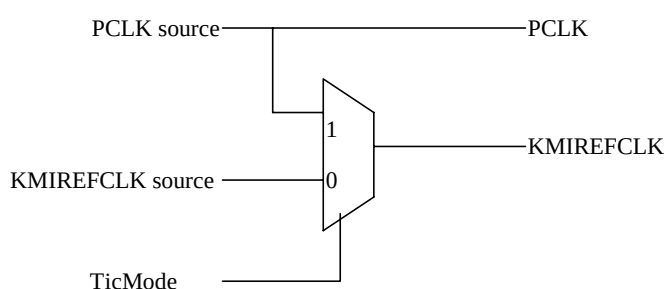


Figure 4.9-1 Multiplexing KMIREFCLK to PCLK source for production TIC testing

For successful clock tree insertion, there must be a clear path from the PCLK source to KMIREFCLK so that the clock trees PCLK and KMIREFCLK will be balanced to each other. If the design flow being used is not capable of supporting balancing of two clock trees then it is probably not possible to use TIC methodology for production testing.

4.10 Clock Tree synthesis

Many different design methodologies are available for clock tree insertion and synthesis. The default KMI synthesis scripts cause the Synopsys tools to output a netlist with no buffering on the clock nets. It is assumed that clock buffering will be performed by a separate tool, such as place & route tools which understand placement, further on in the backend process. It is possible to modify the scripts if this is not the preferred approach or if this is not compatible with the design flow being used. It is recommended however that clock tree insertion should always be performed during the place and route stage and not during synthesis.

4.11 Resets

The KMI module has one reset pin for each of its internal clock domains. The system bus reset signal BnRES, resets the PCLK domain registers. A separate reset pin nKMIRST, is used to reset the KMIREFCLK domain registers. This signal needs to be provided by a separate Reset Controller, and it must be de-asserted synchronously to KMIREFCLK. The resets can be asserted asynchronously even if no clocks are present to ensure that signals from bi-directional I/O cells have defined values during reset, thus avoiding static power consumption problems. While re-synchronisation of BnRES to KMIREFCLK could have been done within the KMI module itself, the function was moved out to eliminate internally generated resets.

Figure 4-11-1 below illustrates a method of generating nKMIRST from BnRES, such that it will be asserted asynchronously and de-asserted synchronous to KMIREFCLK. The reset signal nKMIRST is de-asserted two KMIREFCLK cycles after BnRES is de-asserted to prevent meta-stability problems

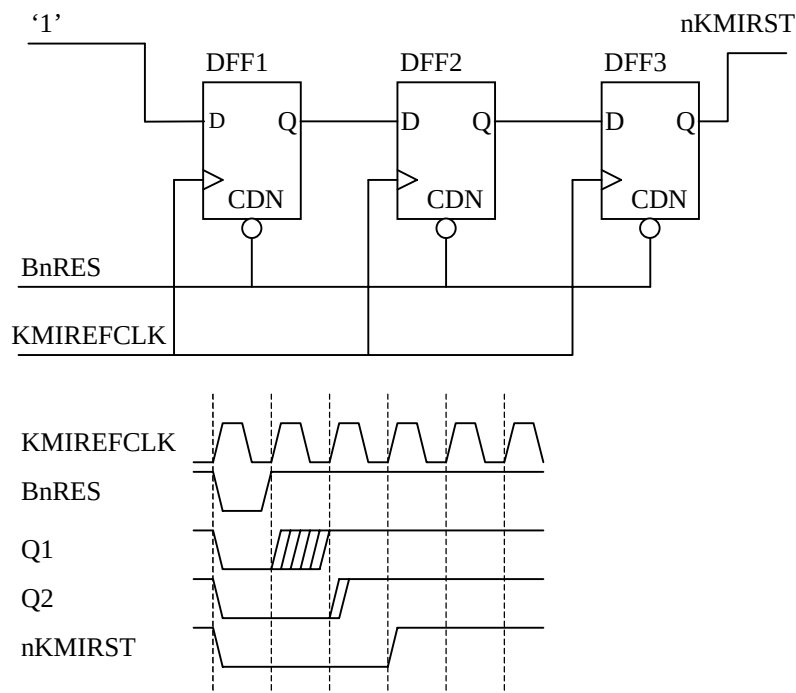


Figure 4.11-1 A method of generating nKMIRST

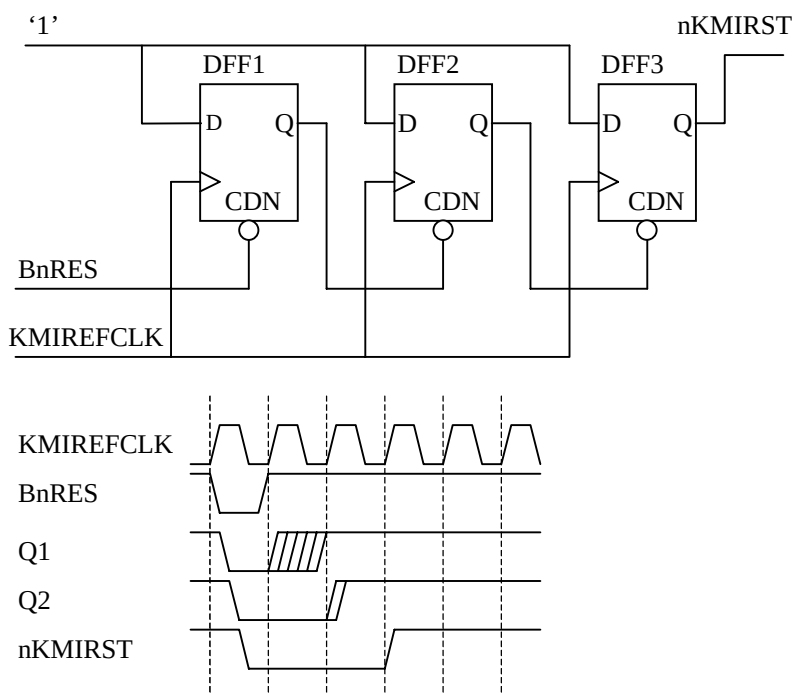


Figure 4.11-2 An alternative method of generating $nKMIRST$

The method of generating $nKMIRST$ in Figure 4.11-1 could experience simulation problems depending on the cell library modelling. The Q output of DFF3 should not be meta-stable when setup/hold violation occurs due to de-assertion of BnRES, since both D and Q are both '0', however the DFF model may assign 'X'.

An alternative method of generating $nKMIRST$ is therefore illustrated in Figure 4.11-2, which achieves the same functionality. This method does use internally generated resets, which can present problems in Static Timing Analysis, scan insertion and ATPG, but they are limited to the reset controller at the top-level of the design hierarchy.

This peripheral contains a small amount of logic in the reset net to implement a test reset signal in order to increase fault coverage when production testing with TicTalk test vectors. The logic in the reset nets is located within a separate hierarchical sub-block KmiTest.

4.12 Reset Buffering

One of several available methods should be used to ensure correct buffering of the reset nets. The reset signals could be assigned infinite drive strength allowing a chip-level integration process to ensure balanced reset trees are used at the top-level of the chip or propagating the tree down to all end points. Another alternative is to apply timing constraints and ensure that block synthesis instantiates cells with adequate drive strength in each block.

The SCANMODE input must be asserted High during scan testing to guarantee that all storage elements will be cleared when the KMI reset inputs are asserted. During normal operation SCANMODE must be de-asserted.

4.13 Synchronisers

All control signals between the two clock domains are synchronised by two series flip-flops inferred during synthesis from standard HDL templates.

Some technology libraries offer meta-stability hardened D flip-flops specifically for implementing synchronisers. All the synchronisers in the KMI block have been isolated to the sub-blocks KmiSyncToPCLK and KmiSyncToREFCLK. This allows the synthesis scripts to be modified so that these blocks could be synthesized separately to use the synchroniser flip-flop cells using the Synopsys Design Compiler `set_prefer` command. Further synthesis of these sub-blocks can then be prevented using the `set_dont_touch` command. The rest of the KMI design is then synthesized after placing a `set_dont_use` command on the synchroniser cells. This method also offers the opportunity to simulate the gate-level netlist with asynchronous clocks PCLK and KMIREFCLK and eliminate the expected setup and hold violations on the synchronisers from the simulation transcript, since the synchronisers cells can be modified to remove 'X' propagation.

4.14 Test Methodologies

The KMI is designed to be tested in one of two ways, depending on the application:

4.14.1 AMBA TIC methodology

AMBA TIC test methodology has the following advantages:

- a) Gate count will be less than if scan insertion is used.
- b) Less steps and tools required in the design flow by not using scan insertion/ATPG. It removes a stage from the design flow since there is no need to modify the synthesised netlist to add scan cells.
- c) The TIC is already present when full-custom ARM cores are used.
- d) TIC testing provides a degree of functional verification and debugging in-situ, and can be used for RTL and gate-level equivalence checking by dynamic simulation.
- e) TIC test vectors require less tester patterns than with ATPG.
- f) TIC testing provides better support for use of non-standard ASIC design rules.

The TIC methodology uses an external 32-bit wide data port and an external 4-bit control interface to gain access to the internal peripheral under test via read/write transfers on the APB. Internal TIC (test) registers in the KMI block allow all the inputs to the peripheral to be stimulated and the outputs to be monitored by the test program. Additionally, test control registers allow some aspects of operation to be put into test modes to reduce the number of vectors required. The TIC methodology is most useful in the case where no modification is required to the KMI design.

To use the TIC methodology, it is necessary for the TIC (Test Interface Controller) module supplied as part of the Micropack product, to be included as a master in the system. Additionally, it is necessary to provide access for the 32 bits of TDATA (normally through the chip's external data bus), the three inputs TCLK, TREQA and TREQB, and the output TACK. Some of these signals could be multiplexed with the General Purpose I/O pins to allow test access. The system test harness should include the external behavioural 'TICBOX' from Micropack, which interprets the Test Input File into a series of commands to the internal TIC block, and applies the appropriate stimulus to the TIC data and control signals.

Assuming a hard macro ARM core is included in the chip, the TIC module and infrastructure is a necessary requirement to allow testing of the ARM core. Therefore, it makes sense to use the infrastructure already present to allow easy testing of the other peripheral blocks as well.

With the KMI block synthesized to a typical standard cell library, the supplied test code can achieve high fault coverage. When retargeted to another cell library, the fault coverage may vary slightly. If the fault coverage is

reduced by an excessive margin, it implies inefficient or redundant logic in the synthesised netlist. The schematic should be inspected in this case. However, test program development may require significant resource and it may not be feasible to achieve a very high coverage figure as large as 99%.

It should also be noted that any change to the design will require corresponding changes to the test compliance program in order to generate new AMBA TIC test vectors, and that this change will alter fault coverage.

4.14.2 Scan Insertion and ATPG

For some designs, it will be preferable to use a scan Insertion and ATPG test methodology. It is assumed that the user will already be familiar with the design flow for scan insertion and ATPG, so only a brief discussion of integration issues arising are given below.

The KMI design fully supports this scan test and ATPG methodology. Depending on the clocking arrangement used (see **section 4.9**), it will be necessary to use either one or two scan chains. Only the rising edges of clocks are used in the design.

If scan insertion is to be performed post-synthesis, it should be done once the netlist has been proven to be logically equivalent either by dynamic simulation using the functional test vectors or use of equivalence checking. Scan insertion is then performed and the logical equivalence again proven. For scan insertion using a two-pass approach, the RTL is compiled into a netlist without scan cells in the first pass and in the second pass, the normal storage elements are replaced with scan cells and the scan chains are formed. Alternatively, a one-pass approach may be adopted for synthesis where the RTL is compiled to create netlist with a scan test D flips-flops and provisionally connected scan chains. After scan insertion logical equivalence should again be proven.

The main advantage of the scan Insertion and ATPG approach is in cases where significant modifications have been made to the KMI module design by the user, or when most other user-generated parts of the chip use scan testing. Another advantage is that a very high fault coverage can be achieved (99%).

If gate count is a critical issue and a scan approach is to be taken, the TIC test registers may be removed from the design. This however, is not recommended, as the design cannot be functionally verified using the TICTalk test vectors supplied.

Extraction of test vectors is done by the system designer once the whole chip design has been integrated. The quoted fault coverage (excepting variation resulting from the use of a sparse cell library) will be met by extracting test vectors around the complete chip while running either the TIC source or the automatically generated vectors (ATPG). In the case of TIC, only the 36 TIC port signals need to be controlled/monitored by the tester to achieve the required fault coverage.

A chip-level test mode is required to force the SCANMODE input high during scan testing to guarantee that the reset inputs will clear all storage elements within each clock domain of the KMI block. This is necessary due to logic being implemented on the reset nets.

The SCANMODE pin must be asserted HIGH during scan testing to guarantee that the BnRES and nKMIRST inputs will clear all storage elements within each clock domain and that the KMITCR test register TESTRST test bit action is disabled when data is scanned through this register.

4.15 Functional and Timing Verification

It is essential to verify timing performance of the netlist against the RTL at key stages in the design flow, post-synthesis, post-scan insertion, during layout and at completion before sign-off of the ASIC.

Gate-level simulation can be used for limited timing verification but it is slow and cannot exercise all timing paths in design. Results tend to be optimistic since worst case timing path may not be exercised.

Static timing verification is a worthwhile and efficient method of verifying timing performance of a design block, but can tend to be pessimistic unless false paths are defined to exclude such paths from analysis. Use of gate-level simulation as well as Static Timing Analysis is a useful check for human error in the manual definition of false paths.

The netlist should also be checked for functional equivalence to the RTL at the same key stages of the design flow.

This can be performed by dynamic simulation using AMBA TIC test vectors as described above, or by use of a logic equivalence checking tool (For example, Synopsys Formality or Chrysalis).